증명 생성 프로세스 (Proof Generation Process)

개요

증명 생성(Proof Generation)은 plonky2에서 가장 핵심적인 과정으로, 회로에 대한 witness를 기반으로 영지식증명을 생성합니다. 이 문서에서는 `prove` 함수의 전체 흐름을 단계별로 분석하고, 각 단계에서 사용되는 핵심 함수와 데이터 구조를 상세히 설명합니다.

1. Witness 생성 과정

1.1 PartialWitness 구조

`PartialWitness` 는 회로의 일부 Target에 대한 값을 저장하는 구조체입니다. 사용자는 회로를 구성한 후, 공개 입력과 비공개 입력에 해당하는 Target들의 값을 설정합니다.

```
#[derive(Clone, Debug, Default)]
pub struct PartialWitness<F: Field> {
pub target_values: HashMap<Target, F>,
}

impl<F: Field> PartialWitness<F> {
pub fn new() -> Self {
Self {
target_values: HashMap::new(),
}
```

```
}

}
```

구조 설명:

- `target_values` : Target을 키로, 필드 원소 값을 값으로 하는 해시맵
- 사용자가 직접 설정한 입력값들만 포함
- 회로 내부의 중간 계산 결과는 포함하지 않음

1.2 set_target 함수 사용법

`set_target` 함수는 특정 Target에 값을 설정하는 메서드입니다.

```
impl<F: Field> WitnessWrite<F> for PartialWitness<F> {

fn set_target(&mut self, target: Target, value: F) -> Result<()> {

let opt_old_value = self.target_values.insert(target, value);

if let Some(old_value) = opt_old_value {

if value != old_value {

return Err(anyhow!(

"Target {:?} was set twice with different values: {} != {}",

target,

old_value,

value

));

}

}
```

```
Ok(())
}
}
```

동작 방식:

1. HashMap에 (target, value) 쌍을 삽입
2. 이미 값이 설정되어 있었다면 이전 값을 반환
3. 이전 값과 새 값이 다르면 에러 발생 (일관성 검증)
4. 같은 값이면 정상 처리

사용 예시:

```
let mut pw = PartialWitness::new();
pw.set_target(initial_a, F::ZERO)?;
pw.set_target(initial_b, F::ONE)?;
```

1.3 generate_partial_witness 함수 분석

`generate_partial_witness` 함수는 사용자가 제공한 `PartialWitness` 를 완전한 witness로 확장합니다. 이 과정에서 Generator들이 실행되어 회로 내부의 모든 Wire 값을 계산합니다.

```
pub fn prove<F: RichField + Extendable<D>, C: GenericConfig<D, F = F>, const D
prover_data: &ProverOnlyCircuitData<F, C, D>,
common_data: &CommonCircuitData<F, D>,
inputs: PartialWitness<F>,
timing: &mut TimingTree,
```

```
) -> Result<ProofWithPublicInputs<F, C, D>>
{
let partition_witness = timed!(
timing,
&format!("run {} generators", prover_data.generators.len()),
generate_partial_witness(inputs, prover_data, common_data)?
);
// ...
}
```

PartitionWitness 구조:

```
#[derive(Clone, Debug)]
pub struct PartitionWitness<'a, F: Field> {
pub values: Vec<Option<F>>,
pub representative_map: &'a [usize],
pub num_wires: usize,
pub degree: usize,
}
```

필드 설명:

- `values` : 각 대표 Target의 값을 저장하는 벡터
- `representative_map` : 각 Target의 대표 인덱스를 가리키는 맵 (Union-Find 구조)
- `num_wires` : 회로의 Wire 개수

- `degree` : 회로의 차수 (게이트 개수)

Witness 생성 과정:

1. **초기화**: `PartialWitness` 의 값들을 `PartitionWitness` 로 복사
2. **Generator 실행**: 각 Generator가 순차적으로 실행되어 추가 Wire 값 계산
3. **Copy Constraint 적용**: 동일한 파티션에 속한 Target들은 같은 값을 가짐
4. **완전성 검증**: 모든 필요한 Wire 값이 설정되었는지 확인

full_witness 변환:

```
pub fn full_witness(self) -> MatrixWitness<F> {
let mut wire_values = vec![vec![F::ZERO; self.degree]; self.num_wires];
for i in 0..self.degree {
for j in 0..self.num_wires {
let t = Target::Wire(Wire { row: i, column: j });
if let Some(x) = self.try_get_target(t) {
wire_values[j][i] = x;
}
}
}
MatrixWitness { wire_values }
}
```

이 함수는 `PartitionWitness` 를 `MatrixWitness` 로 변환합니다:

- `wire_values[j][i]` : j번째 Wire의 i번째 게이트에서의 값
- 행렬 형태로 저장하여 다항식 보간에 용이

2. 증명 생성 메인 로직

2.1 prove 함수 전체 흐름

`prove` 함수는 증명 생성의 핵심 함수로, 다음과 같은 단계로 진행됩니다:

```
pub fn prove<F: RichField + Extendable<D>, C: GenericConfig<D, F = F>, const D
prover_data: &ProverOnlyCircuitData<F, C, D>,
common_data: &CommonCircuitData<F, D>,
inputs: PartialWitness<F>,
timing: &mut TimingTree,
) -> Result<ProofWithPublicInputs<F, C, D>>
```

입력 파라미터:

- `prover_data` : Prover 전용 데이터 (Generator, 전처리된 다항식 등)
- `common_data` : Prover와 Verifier가 공유하는 데이터 (회로 구조, 설정 등)
- `inputs` : 사용자가 제공한 부분 witness
- `timing` : 성능 측정용 타이밍 트리

반환값:

- `ProofWithPublicInputs` : 증명과 공개 입력을 포함하는 구조체

2.2 Wire 값 계산 과정

Witness가 생성되면, 각 Wire의 값을 다항식으로 변환합니다.

```
let witness = timed!(
```

```
timing,
"compute full witness",
partition_witness.full_witness()
);
```

```
let  wires_values: Vec<PolynomialValues<F>> = timed!(
timing,
"compute wire polynomials",
witness
.wire_values
.par_iter()
.map(|column| PolynomialValues::new(column.clone()))
.collect()
);
```

동작 설명:

1. `partition_witness.full_witness()` : PartitionWitness를 MatrixWitness로 변환
2. `witness.wire_values` : 각 Wire별로 모든 게이트에서의 값을 저장한 벡터
3. `par_iter()` : 병렬 처리를 통한 성능 최적화 (Rayon 사용)
4. `PolynomialValues::new()` : 각 Wire의 값들을 다항식 값으로 래핑

결과:

- `wires_values[i]` : i번째 Wire의 모든 게이트에서의 값을 나타내는 다항식
- 각 다항식은 degree개의 점에서 평가된 값을 가짐

2.3 Wire Commitment 생성 (PolynomialBatch::from_values)

Wire 다항식들을 커밋하여 Merkle Tree를 생성합니다.

```
let  wires_commitment = timed!(
timing,
"compute wires commitment",
PolynomialBatch::<F, C, D>::from_values(
wires_values,
config.fri_config.rate_bits,
config.zero_knowledge && PlonkOracle::WIRES.blinding,
config.fri_config.cap_height,
timing,
prover_data.fft_root_table.as_ref(),
)
);
```

파라미터 설명:

- `wires_values` : Wire 다항식들의 값
- `rate_bits` : FRI 프로토콜의 rate (LDE 확장 비율)
- `blinding` : Zero-knowledge를 위한 블라인딩 여부
- `cap_height` : Merkle Tree의 cap 높이
- `fft_root_table` : FFT 최적화를 위한 사전 계산된 루트 테이블

PolynomialBatch::from_values 내부 동작:

1. **다항식 보간 (Interpolation)**:

- 각 Wire의 값들을 다항식 계수로 변환
- IFFT (Inverse FFT)를 사용하여 효율적으로 계산

2. **Low Degree Extension (LDE)**:

- 다항식을 더 큰 도메인으로 확장
- 확장 비율 = 2^rate_bits
- FFT를 사용하여 확장된 도메인에서 평가

3. **Merkle Tree 구성**:

- LDE 값들을 리프 노드로 하는 Merkle Tree 생성
- 각 리프는 여러 다항식의 값을 포함 (batching)
- 해시 함수를 사용하여 트리 구성

4. **Blinding (선택적)**:

- Zero-knowledge를 위해 랜덤 다항식 추가
- 증명에서 witness 정보가 누출되지 않도록 보호

결과 구조:

```
pub struct PolynomialBatch<F, C, D> {
pub polynomials: Vec<PolynomialCoeffs<F>>,
pub merkle_tree: MerkleTree<F, C::Hasher>,
pub degree_log: usize,
pub rate_bits: usize,
}
```

- `polynomials` : 다항식 계수들

- `merkle_tree` : 구성된 Merkle Tree (cap 포함)
- `degree_log` : log2(degree)
- `rate_bits` : LDE 확장 비율

2.4 FFT 및 Merkle Tree 구성 과정

2.4.1 FFT (Fast Fourier Transform)

FFT는 다항식 연산을 효율적으로 수행하기 위한 핵심 알고리즘입니다.

IFFT (Inverse FFT) - 보간:

```
입력: 점 평가 값 [f(ω^0), f(ω^1), ..., f(ω^(n-1))]

출력: 다항식 계수 [a_0, a_1, ..., a_(n-1)]


여기서 f(x) = a_0 + a_1*x + ... + a_(n-1)*x^(n-1)
```

FFT - 평가:

```
입력: 다항식 계수 [a_0, a_1, ..., a_(n-1)]

출력: 점 평가 값 [f(ω^0), f(ω^1), ..., f(ω^(n-1))]
```

plonky2의 FFT 최적화:

- Cache-friendly 메모리 접근 패턴
- Bit-reversal 최적화
- 사전 계산된 루트 테이블 사용 (`fft_root_table`)
- 병렬 처리 (Rayon)

2.4.2 Merkle Tree 구성

Merkle Tree는 다항식 커밋먼트를 생성하는 데 사용됩니다.

구조:

```
Root

/ \

/ \

Hash1 Hash2

/ \ / \

/ \ / \

Hash3 Hash4 Hash5 Hash6

/ \ / \ / \ / \

L0 L1 L2 L3 L4 L5 L6 L7
```

Cap (Merkle Cap):

- Merkle Tree의 상위 일부 레벨
- `cap_height` 로 지정된 높이까지의 노드들
- 증명 크기와 검증 시간의 트레이드오프

예시:

```
// cap_height = 2인 경우

// Cap = [Hash1, Hash2]

// 이 두 해시만으로 전체 트리를 대표
```

Merkle Tree 생성 과정:

1. **리프 노드 생성**:

```
// 각 리프는 여러 다항식의 LDE 값을 포함
leaf[i] = Hash(poly1[i] || poly2[i] || ... || polyN[i])
```

2. **상위 노드 계산**:

```
// 두 자식 노드를 해싱하여 부모 노드 생성
parent = Hash(left_child || right_child)
```

3. **Cap 추출**:

```
// cap_height 레벨의 모든 노드를 Cap으로 저장
cap = nodes_at_level(cap_height)
```

Merkle Proof:

- 특정 리프의 포함 증명
- 리프에서 Cap까지의 경로상의 형제 노드들
- 크기: O(log(n) - cap_height)

3. Fiat-Shamir 변환 및 챌린지 생성

3.1 Challenger 구조 설명

`Challenger` 는 Fiat-Shamir 변환을 구현하는 구조체로, 증명자의 메시지를 관찰하고 검증자의 챌린지를 생성합니다.

```
#[derive(Clone, Debug)]
```

```rust
pub struct Challenger<F: RichField, H: Hasher<F>> {
pub(crate) sponge_state: H::Permutation,
pub(crate) input_buffer: Vec<F>,
output_buffer: Vec<F>,
}
```

필드 설명:

- `sponge_state` : 스펀지 함수의 내부 상태 (해시 함수 기반)
- `input_buffer` : 아직 처리되지 않은 입력들을 버퍼링
- `output_buffer` : 생성된 챌린지들을 버퍼링

Fiat-Shamir 변환이란?

대화형 프로토콜을 비대화형으로 변환하는 기법:

- **대화형**: Prover ↔ Verifier가 여러 라운드 통신
- **비대화형**: Prover가 증명을 한 번에 생성, Verifier가 검증

변환 방법:

1. Verifier의 랜덤 챌린지를 해시 함수로 대체
2. 증명자의 이전 메시지들을 해시하여 챌린지 생성
3. 이를 통해 증명자가 챌린지를 미리 알 수 없도록 보장

3.2 Challenger 초기화 및 사용

```rust
let mut challenger = Challenger::<F, C::Hasher>::new();

// FRI 설정 관찰
```

```
common_data.fri_params.observe(&mut challenger);


// 회로 다이제스트 관찰
challenger.observe_hash::<C::Hasher>(prover_data.circuit_digest);


// 공개 입력 해시 관찰
challenger.observe_hash::<C::InnerHasher>(public_inputs_hash);


// Wire commitment의 Merkle Cap 관찰
challenger.observe_cap::<C::Hasher>(&wires_commitment.merkle_tree.cap);
```

관찰 순서의 중요성:

- 모든 증명자와 검증자가 동일한 순서로 관찰해야 함
- 순서가 다르면 다른 챌린지가 생성되어 검증 실패
- 프로토콜의 보안성을 위해 모든 공개 정보를 관찰

3.3 observe_cap 함수 분석

`observe_cap` 함수는 Merkle Cap을 Challenger에 입력합니다.

```
pub fn observe_cap<OH: Hasher<F>>(&mut self, cap: &MerkleCap<F, OH>) {
for &hash in &cap.0 {
self.observe_hash::<OH>(hash);
}
```

```
}
```

동작:

1. Cap의 각 해시를 순회
2. 각 해시를 `observe_hash` 로 관찰
3. 해시의 모든 원소를 `observe_element` 로 입력

observe_element 내부:

```
pub fn observe_element(&mut self, element: F) {
// 버퍼링된 출력은 이제 무효화됨
self.output_buffer.clear();
self.input_buffer.push(element);
// 버퍼가 가득 차면 duplexing 수행
if self.input_buffer.len() == H::Permutation::RATE {
self.duplexing();
}
}
```

Duplexing (이중화):

```
fn duplexing(&mut self) {
// 입력 버퍼의 내용을 스펀지 상태에 덮어쓰기
self.sponge_state.set_from_iter(self.input_buffer.drain(..), 0);
// 순열 함수 적용 (해시 함수의 핵심)
self.sponge_state.permute();
```

```
// 출력 버퍼를 스펀지 상태에서 추출
self.output_buffer.clear();
self.output_buffer.extend_from_slice(self.sponge_state.squeeze());
}
```

Duplex Sponge 구조:

```
입력 → [Rate | Capacity] → Permutation → [Rate | Capacity] → 출력
↑ ↓
└──────────────── 피드백 ────────────────┘
```

- **Rate**: 입출력이 가능한 부분
- **Capacity**: 보안을 위해 숨겨진 부분
- **Permutation**: 암호학적 순열 함수 (예: Poseidon)

3.4 get_n_challenges 함수 분석

챌린지를 생성하는 함수입니다.

```
pub fn get_n_challenges(&mut self, n: usize) -> Vec<F> {
(0..n).map(|_| self.get_challenge()).collect()
}

pub fn get_challenge(&mut self) -> F {
// 버퍼링된 입력이 있거나 출력이 없으면 duplexing 수행
```

```
if !self.input_buffer.is_empty() || self.output_buffer.is_empty() {
self.duplexing();
}
self.output_buffer
.pop()
.expect("Output buffer should be non-empty")
}
```

동작 방식:

1. 출력 버퍼가 비어있으면 duplexing 수행
2. 출력 버퍼에서 원소를 하나씩 꺼내 반환
3. 필요한 만큼 반복

3.5 Beta, Gamma 챌린지 생성

증명 생성 과정에서 여러 챌린지가 생성됩니다.

```
let num_challenges = config.num_challenges;

// Beta 챌린지 생성
let betas = challenger.get_n_challenges(num_challenges);

// Gamma 챌린지 생성
let gammas = challenger.get_n_challenges(num_challenges);
```

챌린지의 역할:

Beta (β):

- Permutation Argument에서 사용
- Wire 값과 결합하여 랜덤화
- 각 Wire에 대해 `wire_value + β * s_id + γ` 형태로 사용

Gamma (γ):

- Permutation Argument에서 추가 랜덤화
- Beta와 함께 사용하여 보안성 강화
- Zero-knowledge 속성 보장

Delta (δ) - Lookup용:

```
let deltas = if has_lookup {
let mut delts = Vec::with_capacity(2 * num_challenges);
let num_additional_challenges = num_lookup_challenges - 2 * num_challenges;
let additional = challenger.get_n_challenges(num_additional_challenges);
delts.extend(&betas);
delts.extend(&gammas);
delts.extend(additional);
delts
} else {
vec![]
};
```

- Lookup 테이블이 있는 경우에만 생성
- Beta, Gamma를 재사용하고 추가 챌린지 생성

- Lookup Argument에서 사용

챌린지 생성 타이밍:

```
1. Wire Commitment 생성

2. Wire Cap 관찰

3. Beta, Gamma 챌린지 생성 ← 여기

4. Z 다항식 계산 (Beta, Gamma 사용)

5. Z Commitment 생성

6. Z Cap 관찰

7. Alpha 챌린지 생성

8. Quotient 다항식 계산 (Alpha 사용)

...
```

보안 고려사항:

- 각 챌린지는 이전 단계의 모든 정보를 반영
- 증명자가 챌린지를 미리 알 수 없음
- 충분한 엔트로피 확보 (num_challenges 개수)

4. Permutation Argument (순열 논증)

4.1 Z 다항식의 수학적 정의

Permutation Argument는 PLONK의 핵심 메커니즘으로, Copy Constraint를 검증합니다.

목적:

- 서로 다른 위치의 Wire들이 같은 값을 가져야 함을 증명

- 예: `a = b, b = c → a = c` (전이성)

Z 다항식의 정의:

Z 다항식은 다음 재귀 관계를 만족합니다:

```
Z(1) = 1
```

```
Z(g^(i+1)) = Z(g^i) * Π[j=0 to k-1] (w_j(g^i) + β·s_id_j(g^i) + γ)
/ (w_j(g^i) + β·s_σ_j(g^i) + γ)
```

```
Z(g^n) = 1 (순환 조건)
```

여기서:

- `g` : 생성원 (generator)
- `n` : 회로의 차수 (degree)
- `k` : 라우팅된 Wire의 개수
- `w_j` : j번째 Wire의 값
- `s_id_j` : Identity permutation (k_j · x)
- `s_σ_j` : Sigma permutation (실제 순열)
- `β, γ` : 챌린지 값

직관적 이해:

분자(Numerator)와 분모(Denominator)의 곱:

- **분자**: Wire 값들이 Identity permutation을 따를 때의 곱
- **분모**: Wire 값들이 실제 Sigma permutation을 따를 때의 곱

두 순열이 같은 multiset을 나타내면 Z(g^n) = 1이 됩니다.

4.2 all_wires_permutation_partial_products 함수 분석

```
fn  all_wires_permutation_partial_products<
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F>,
const D: usize,
>(
witness: &MatrixWitness<F>,
betas: &[F],
gammas: &[F],
prover_data: &ProverOnlyCircuitData<F, C, D>,
common_data: &CommonCircuitData<F, D>,
) -> Vec<Vec<PolynomialValues<F>>> {
(0..common_data.config.num_challenges)
.map(|i| {
wires_permutation_partial_products_and_zs(
witness,
betas[i],
gammas[i],
prover_data,
common_data,
)
})
```

```
.collect()
}
```

동작:

- 각 챌린지에 대해 독립적으로 Z 다항식 계산
- `num_challenges` 개의 Z 다항식 생성 (보안성 향상)
- 병렬 처리 가능

4.3 Partial Products 계산 과정

```
fn  wires_permutation_partial_products_and_zs<
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F>,
const D: usize,
>(
witness: &MatrixWitness<F>,
beta: F,
gamma: F,
prover_data: &ProverOnlyCircuitData<F, C, D>,
common_data: &CommonCircuitData<F, D>,
) -> Vec<PolynomialValues<F>>
```

단계별 계산:

1. 각 게이트에서 Quotient 계산:

```
let all_quotient_chunk_products = subgroup
.par_iter()
.enumerate()
.map(|(i, &x)| {
let s_sigmas = &prover_data.sigmas[i];
// 분자 계산
let numerators = (0..common_data.config.num_routed_wires).map(|j| {
let wire_value = witness.get_wire(i, j);
let k_i = k_is[j];
let s_id = k_i * x;
wire_value + beta * s_id + gamma
});
// 분모 계산
let denominators = (0..common_data.config.num_routed_wires)
.map(|j| {
let wire_value = witness.get_wire(i, j);
let s_sigma = s_sigmas[j];
wire_value + beta * s_sigma + gamma
})
.collect::<Vec<_>>();
// 역원 계산 (배치 최적화)
let denominator_invs = F::batch_multiplicative_inverse(&denominators);
```

```
// Quotient 값 계산
let  quotient_values = numerators
.zip(denominator_invs)
.map(|(num, den_inv)| num * den_inv)
.collect::<Vec<_>>();
quotient_chunk_products(&quotient_values, degree)
})
.collect::<Vec<_>>();
```

Quotient 값의 의미:

```
quotient[j] = (w_j + β·s_id_j + γ) / (w_j + β·s_σ_j + γ)
```

2. Partial Products 계산:

`quotient_chunk_products` 함수는 quotient 값들을 청크로 나누어 부분 곱을 계산합니다:

```
degree = quotient_degree_factor

chunk_0 = quotient[0] * quotient[1] * ... * quotient[degree-1]
chunk_1 = quotient[degree] * quotient[degree+1] * ... * quotient[2*degree-1]
...
```

이는 Z 다항식의 차수를 제한하기 위한 최적화입니다.

3. Z 다항식 누적 계산:

```
let mut z_x = F::ONE;
let mut all_partial_products_and_zs = Vec::with_capacity(all_quotient_chunk_pr

for quotient_chunk_products in all_quotient_chunk_products {
let mut partial_products_and_z_gx =
partial_products_and_z_gx(z_x, &quotient_chunk_products);
// 마지막 항은 Z(gx)이지만, Z(x)로 교체
swap(&mut z_x, &mut partial_products_and_z_gx[num_prods]);
all_partial_products_and_zs.push(partial_products_and_z_gx);
}
```

partial_products_and_z_gx 함수:

```
pub fn partial_products_and_z_gx<F: Field>(
z_x: F,
quotient_chunk_products: &[F],
) -> Vec<F> {
let mut result = Vec::with_capacity(quotient_chunk_products.len() + 1);
let mut acc = z_x;
for &chunk_product in quotient_chunk_products {
result.push(acc);
acc *= chunk_product;
}
```

```
result.push(acc); // Z(gx)

result

}
```

결과:

```
[Z(x), Z(x)·chunk_0, Z(x)·chunk_0·chunk_1, ..., Z(gx)]
```

4. 다항식 변환:

```
transpose(&all_partial_products_and_zs)

.into_par_iter()

.map(PolynomialValues::new)

.collect()
```

- Transpose: 행렬 전치 (게이트별 → 다항식별)
- 각 열이 하나의 다항식이 됨
- 마지막 다항식이 Z 다항식

최종 결과:

```
let mut partial_products_and_zs = all_wires_permutation_partial_products(...);


// Z는 맨 앞에 위치해야 함

let plonk_z_vecs = partial_products_and_zs

.iter_mut()
```

```
.map(|partial_products_and_z| partial_products_and_z.pop().unwrap())
.collect();

let  zs_partial_products = [plonk_z_vecs, partial_products_and_zs.concat()].conc
```

구조:

```
[Z_0, Z_1, ..., Z_(num_challenges-1),
PP_0_0, PP_0_1, ..., PP_0_k,
PP_1_0, PP_1_1, ..., PP_1_k,
...]
```

여기서:

- `Z_i` : i번째 챌린지에 대한 Z 다항식
- `PP_i_j` : i번째 챌린지, j번째 부분 곱 다항식

5. Quotient Polynomial 계산

5.1 Vanishing Polynomial 개념 설명

Vanishing Polynomial (소멸 다항식):

Vanishing Polynomial Z_H(x)는 부분군 H의 모든 원소에서 0이 되는 다항식입니다:

```
Z_H(x) = x^n - 1
```

```
여기서 H = {1, g, g^2, ..., g^(n-1)}
```

성질:

- `Z_H(g^i) = 0` for all i ∈ [0, n-1]
- `Z_H(x) ≠ 0` for x ∉ H

PLONK에서의 역할:

모든 제약조건은 부분군 H에서만 만족되면 됩니다. 따라서:

```
제약조건(x) = 0 for all x ∈ H

⇔

제약조건(x) = Z_H(x) · Q(x) for some polynomial Q(x)
```

Q(x)를 Quotient Polynomial이라고 합니다.

5.2 Alpha 챌린지 생성

Z 다항식 커밋먼트 후 Alpha 챌린지를 생성합니다:

```
// Z와 Partial Products 커밋
let  partial_products_zs_and_lookup_commitment = timed!(
timing,
"commit to partial products, Z's and, if any, lookup polynomials",
PolynomialBatch::from_values(
zs_partial_products_lookups,
config.fri_config.rate_bits,
config.zero_knowledge && PlonkOracle::ZS_PARTIAL_PRODUCTS.blinding,
```

```
config.fri_config.cap_height,
timing,
prover_data.fft_root_table.as_ref(),
)
);

// Cap 관찰
challenger.observe_cap::<C::Hasher>(
&partial_products_zs_and_lookup_commitment.merkle_tree.cap
);

// Alpha 챌린지 생성
let  alphas = challenger.get_n_challenges(num_challenges);
```

Alpha의 역할:

- 여러 제약조건을 하나의 다항식으로 결합
- 각 제약조건에 랜덤 가중치 부여
- 보안성 유지하면서 증명 크기 감소

5.3 compute_quotient_polys 함수 분석

```
let  quotient_polys = timed!(
timing,
"compute quotient polys",
```

```
compute_quotient_polys::<F, C, D>(
common_data,
prover_data,
&public_inputs_hash,
&wires_commitment,
&partial_products_zs_and_lookup_commitment,
&betas,
&gammas,
&deltas,
&alphas,
)
);
```

함수 시그니처:

```
fn  compute_quotient_polys<
'a,
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F>,
const D: usize,
>(
common_data: &CommonCircuitData<F, D>,
prover_data: &'a ProverOnlyCircuitData<F, C, D>,
```

```
public_inputs_hash: &<<C as GenericConfig<D>>::InnerHasher as Hasher<F>>::Hash,
wires_commitment: &'a PolynomialBatch<F, C, D>,
zs_partial_products_and_lookup_commitment: &'a PolynomialBatch<F, C, D>,
betas: &[F],
gammas: &[F],
deltas: &[F],
alphas: &[F],
) -> Vec<PolynomialCoeffs<F>>
```

5.4 Quotient 계산 상세 과정

1. LDE 준비:

```
let quotient_degree_bits = log2_ceil(common_data.quotient_degree_factor);
let step = 1 << (common_data.config.fri_config.rate_bits - quotient_degree_bits
let next_step = 1 << quotient_degree_bits;

let points = F::two_adic_subgroup(common_data.degree_bits() + quotient_degree_b
let lde_size = points.len();
```

- `step` : LDE에서 샘플링 간격
- `next_step` : "다음" 점까지의 거리
- `points` : 확장된 도메인의 점들

2. Vanishing Polynomial on Coset:

```
let z_h_on_coset = ZeroPolyOnCoset::new(
common_data.degree_bits(),
quotient_degree_bits
);
```

Coset에서 Z_H(x) 값을 효율적으로 계산하기 위한 구조체입니다.

3. 배치 처리로 제약조건 평가:

```
const BATCH_SIZE: usize = 32;


let quotient_values: Vec<Vec<F>> = points.par_chunks(BATCH_SIZE)
.enumerate()
.flat_map(|(batch_i, xs_batch)| {
// 각 배치에 대해 제약조건 평가
let vars_batch = EvaluationVarsBaseBatch::new(
xs_batch.len(),
&local_constants_batch,
&local_wires_batch,
public_inputs_hash,
);
let mut quotient_values_batch = eval_vanishing_poly_base_batch::<F, D>(
common_data,
&indices_batch,
```

```
&shifted_xs_batch,
vars_batch,
&local_zs_batch,
&next_zs_batch,
&local_lookup_batch,
&next_lookup_batch,
&partial_products_batch,
&s_sigmas_batch,
betas,
gammas,
deltas,
alphas,
&z_h_on_coset,
&lut_re_poly_evals_refs,
);
// Z_H(x)로 나누기
for (&i, quotient_values) in  indices_batch.iter()
.zip(quotient_values_batch.iter_mut())
{
let  denominator_inv = z_h_on_coset.eval_inverse(i);
quotient_values
.iter_mut()
.for_each(|v| *v *= denominator_inv);
```

```
}
quotient_values_batch
})
.collect();
```

eval_vanishing_poly_base_batch의 역할:

모든 제약조건을 평가하여 결합합니다:

1. **Gate 제약조건**: 각 게이트의 산술 제약
2. **Permutation 제약조건**: Z 다항식 관련 제약
3. **Lookup 제약조건**: Lookup 테이블 제약 (있는 경우)

제약조건 결합:

```
vanishing_poly(x) = α^0 · constraint_0(x)
+ α^1 · constraint_1(x)
+ α^2 · constraint_2(x)
+ ...
```

Quotient 계산:

```
Q(x) = vanishing_poly(x) / Z_H(x)
```

4. 다항식 변환 및 청킹:

```
// Transpose하여 다항식별로 정리
transpose(&quotient_values)
```

```
.into_par_iter()
.map(PolynomialValues::new)
// Coset IFFT로 계수 형태로 변환
.map(|values| values.coset_ifft(F::coset_shift()))
.collect()
```

5. Quotient 다항식 분할:

```
let  all_quotient_poly_chunks: Vec<PolynomialCoeffs<F>> = timed!(
timing,
"split up quotient polys",
quotient_polys
.into_par_iter()
.flat_map(|mut  quotient_poly| {
// Quotient 차수 검증
quotient_poly.trim_to_len(quotient_degree).expect(
"Quotient has failed, the vanishing polynomial is not divisible by Z_H",
);
// degree-n 크기의 청크로 분할
quotient_poly.chunks(degree)
})
.collect()
);
```

분할 이유:

Quotient 다항식의 차수가 너무 높으면:

- 커밋먼트 크기 증가
- 검증 시간 증가

따라서 degree-n 크기로 분할하여:

```
Q(x) = Q_0(x) + x^n · Q_1(x) + x^(2n) · Q_2(x) + ...
```

6. Quotient Commitment:

```
let  quotient_polys_commitment = timed!(
timing,
"commit to quotient polys",
PolynomialBatch::<F, C, D>::from_coeffs(
all_quotient_poly_chunks,
config.fri_config.rate_bits,
config.zero_knowledge && PlonkOracle::QUOTIENT.blinding,
config.fri_config.cap_height,
timing,
prover_data.fft_root_table.as_ref(),
)
);
```

```
challenger.observe_cap::<C::Hasher>(&quotient_polys_commitment.merkle_tree.cap);
```

Quotient 다항식들을 커밋하고 Cap을 Challenger에 관찰시킵니다.

6. FRI Opening Proof

6.1 Opening Set 구성

Quotient 커밋먼트 후, 검증자가 선택한 랜덤 점에서 다항식들을 평가합니다.

```
// Zeta 챌린지 생성
let  zeta = challenger.get_extension_challenge::<D>();


// Zeta가 부분군에 속하지 않는지 검증
let  g = F::Extension::primitive_root_of_unity(common_data.degree_bits());
ensure!(
zeta.exp_power_of_2(common_data.degree_bits()) != F::Extension::ONE,
"Opening point is in the subgroup."
);


// Opening Set 구성
let  openings = timed!(
timing,
"construct the opening set, including lookups",
OpeningSet::new(
```

```
zeta,
g,
&prover_data.constants_sigmas_commitment,
&wires_commitment,
&partial_products_zs_and_lookup_commitment,
&quotient_polys_commitment,
common_data
)
);
```

Opening Set이란?

특정 점(들)에서 여러 다항식의 평가값 집합:

```
pub struct OpeningSet<F: Field, D: usize> {
pub constants: Vec<F::Extension>,
pub plonk_sigmas: Vec<F::Extension>,
pub wires: Vec<F::Extension>,
pub plonk_zs: Vec<F::Extension>,
pub plonk_zs_next: Vec<F::Extension>,
pub partial_products: Vec<F::Extension>,
pub quotient_polys: Vec<F::Extension>,
pub lookup_zs: Vec<F::Extension>,
pub lookup_zs_next: Vec<F::Extension>,
```

```
}
```

평가 점:

- `zeta (ζ)` : 대부분의 다항식 평가 점
- `g * zeta` : Z 다항식의 "다음" 값 평가 점

왜 두 점이 필요한가?

Z 다항식의 재귀 관계를 검증하기 위해:

```
Z(g·ζ) = Z(ζ) · [분자/분모]
```

6.2 prove_openings 함수 분석

```
let opening_proof = timed!(
timing,
"compute opening proofs",
PolynomialBatch::<F, C, D>::prove_openings(
&instance,
&[
&prover_data.constants_sigmas_commitment,
&wires_commitment,
&partial_products_zs_and_lookup_commitment,
&quotient_polys_commitment,
],
&mut challenger,
```

```
&common_data.fri_params,
None,
None,
timing,
)
);
```

입력 파라미터:

- `instance` : FRI 인스턴스 정보 (평가 점, 다항식 정보)
- 다항식 배치들: 증명할 모든 커밋먼트
- `challenger` : Fiat-Shamir 챌린지 생성기
- `fri_params` : FRI 프로토콜 파라미터

FRI Instance 구성:

```
pub(crate) fn get_fri_instance(&self, zeta: F::Extension) -> FriInstanceInfo<F,
// 모든 다항식은 zeta에서 평가
let zeta_batch = FriBatchInfo {
point: zeta,
polynomials: self.fri_all_polys(),
};
// Z 다항식들은 g * zeta에서도 평가
let g = F::Extension::primitive_root_of_unity(self.degree_bits());
let zeta_next = g * zeta;
```

```
let zeta_next_batch = FriBatchInfo {
point: zeta_next,
polynomials: self.fri_next_batch_polys(),
};
let openings = vec![zeta_batch, zeta_next_batch];
FriInstanceInfo {
oracles: self.fri_oracles(),
batches: openings,
}
}
```

6.3 FRI 프로토콜 연결 설명

FRI의 목적:

다항식이 낮은 차수(low degree)임을 증명합니다.

왜 필요한가?

1. 증명자가 정직하게 다항식을 구성했는지 확인
2. 높은 차수 다항식으로 속이는 것을 방지
3. Reed-Solomon 코드의 근접성 검증

FRI 프로토콜 단계:

1. **Commit Phase**:

- 다항식을 반복적으로 폴딩(folding)
- 각 단계마다 Merkle Tree 커밋
- 차수가 충분히 작아질 때까지 반복

2. **Query Phase**:

- 검증자가 랜덤 인덱스 선택
- 증명자가 해당 위치의 값과 Merkle Proof 제공
- 여러 쿼리로 soundness 보장

plonky2에서의 통합:

```
pub fn prove_openings(
instance: &FriInstanceInfo<F, D>,
batches: &[&PolynomialBatch<F, C, D>],
challenger: &mut Challenger<F, C::Hasher>,
fri_params: &FriParams,
// ...
) -> FriProof<F, C::Hasher, D>
```

이 함수는:

1. 모든 다항식 배치를 하나의 다항식으로 결합
2. FRI Commit Phase 실행
3. FRI Query Phase 실행
4. FRI Proof 반환

6.4 최종 증명 구성

모든 단계가 완료되면 최종 증명을 구성합니다:

```
let proof = Proof::<F, C, D> {
```

```
wires_cap: wires_commitment.merkle_tree.cap,

plonk_zs_partial_products_cap: partial_products_zs_and_lookup_commitment.merkle_

quotient_polys_cap: quotient_polys_commitment.merkle_tree.cap,

openings,

opening_proof,

};


Ok(ProofWithPublicInputs::<F, C, D> {

proof,

public_inputs,

})
```

Proof 구조:

```
pub struct Proof<F, C, D> {

/// Wire 다항식들의 Merkle Cap

pub wires_cap: MerkleCap<F, C::Hasher>,

/// Z와 Partial Products 다항식들의 Merkle Cap

pub plonk_zs_partial_products_cap: MerkleCap<F, C::Hasher>,

/// Quotient 다항식들의 Merkle Cap

pub quotient_polys_cap: MerkleCap<F, C::Hasher>,

/// 평가 점에서의 다항식 값들

pub openings: OpeningSet<F, D>,

/// FRI Opening Proof
```

```
pub opening_proof: FriProof<F, C::Hasher, D>,
}
```

ProofWithPublicInputs:

```
pub struct ProofWithPublicInputs<F, C, D> {
pub proof: Proof<F, C, D>,
pub public_inputs: Vec<F>,
}
```

공개 입력과 함께 증명을 패키징합니다.

7. 증명 생성 전체 흐름 요약

```
1. Witness 생성
├─ PartialWitness 입력
├─ Generator 실행
└─ MatrixWitness 생성

2. Wire Commitment
├─ 다항식 보간 (IFFT)
├─ LDE (FFT)
└─ Merkle Tree 구성
```

3. Fiat-Shamir (Beta, Gamma)

└ Wire Cap 관찰 → 챌린지 생성

4. Permutation Argument

├ Z 다항식 계산

├ Partial Products 계산

└ Commitment

5. Fiat-Shamir (Alpha)

└ Z Cap 관찰 → 챌린지 생성

6. Quotient Polynomial

├ 제약조건 평가

├ Z_H로 나누기

├ 청킹

└ Commitment

7. Fiat-Shamir (Zeta)

└ Quotient Cap 관찰 → 챌린지 생성

8. Opening Set 구성

├ Zeta에서 평가

└ g*Zeta에서 평가

```
9. FRI Opening Proof
├─ FRI Commit Phase
├─ FRI Query Phase
└─ FRI Proof 생성

10. 최종 증명 반환
```

이로써 증명 생성 프로세스가 완료됩니다. 생성된 증명은 검증자에게 전달되어 검증 프로세스를 거치게 됩니다.

관련 문서

- 02. 회로 구성 프로세스 - 증명 생성의 전제 조건
- 04. 증명 검증 프로세스 - 생성된 증명의 검증
- 05. FRI 프로토콜 - Opening Proof의 핵심 메커니즘
- 06. Goldilocks Field - 필드 연산의 기초

문서 네비게이션

- **이전**: 02. 회로 구성 프로세스 - CircuitBuilder 사용법
- **다음**: 04. 증명 검증 프로세스 - 증명 검증 방법
- **관련 문서**:
- 05. FRI 프로토콜 - FRI Opening Proof 상세

- 01. 아키텍처 개요 - 전체 데이터 흐름
- 11. 성능 최적화 - 증명 생성 최적화